

Project.1

MapleLike



01

[Unity 2D]

메이플스토리 모작 멀티플레이어 게임

MapleLike

[Unity 2D] 메이플스토리 모작 멀티플레이어 게임

기간

2024. 12 ~ 2025. 03

주관

실전 Unity 게임 개발 (멋쟁이 사자처럼)

참여인원

5명 (서버 개발자 1명 + 클라이언트 개발자 4명)

개발도구

언어	C#
클라이언트 (엔진)	Unity 6000.0.38f1
서버	Protobuf, AWS
협업	Discord, Notion, Plastic SCM

참고 링크 (코드, 협업, 플레이영상)



프로젝트 동기

MMORPG 게임인 메이플스토리를 모작하여 유니티 엔진을 통한 게임 개발 프로세스의 이해 및 **서버 개발자와 클라이언트 개발자 사이의 협업 프로세스를 이해**하기 위해 진행한 프로젝트입니다.

기여 및 담당



- 1. 몬스터 (일반 / 보스) 기획 및 구현
- 2. 맵 디자인 및 플레이어의 맵 이동 구현
- 3. 퀘스트 시스템 구현

회고 및 느낀점

- 1. 멀티플레이어 게임 개발 과정에서, 네트워크 통신이 필요한 기능 구현을 위해 **서버 개발자에게 API 개발을 요청하고 이를 활용하며, 클라이언트-서버 간 협업 프로세스에 대한 이해**를 높일 수 있었습니다.
- 2. 서버와 클라이언트의 기능 구현은 별개의 영역이 아닌 상호 연관된 작업임을 깨달았고, **팀 내 및 팀 간의 원활한 의사소통이 중요함**을 실감했습니다.
- 3. 플레이어와 상호작용하는 게임 내 오브젝트 개발에 있어서, **오브젝트의 라이프 사이클의 이해**가 중요함을 깨달을 수 있었습니다.
- 4. 멀티플레이어 게임에서 플레이어가 긍정적인 경험을 얻기 위해서는 콘텐츠의 퀄리티뿐만 아니라, **서버와 클라이언트 간의 원활한 동기화와 안정성도 중요한 요소**임을 알게 되었습니다.

우수 프로젝트 선정



대상

과정명 : 실전 Unity 게임 개발: 모바일 콘텐츠 제작 기술 과정(2회차)

기간 : 2024.09.23.~2025.03.21.

이름 : 김기환

위 사람은 6개월간 진행된 제 2회차 '과정명: 실전 Unity 게임 개발: 모바일 콘텐츠 제작 기술 과정' 에서 전반적인 태도, 참여도, 협업, 기여도, 프로젝트 결과물 등에서 우수한 성과를 이루어 이 상장을 수여합니다.

2025년 3월 21일

MapleLike

[Unity 2D] 메이플스토리 모작 멀티플레이어 게임

기간

2024. 12 ~ 2025. 03

주관

실전 Unity 게임 개발 (멋쟁이 사자처럼)

참여인원

5명 (서버 개발자 1명 + 클라이언트 개발자 4명)

개발도구

언어	C#
클라이언트 (엔진)	Unity 6000.0.38f1
서버	Protobuf, AWS
협업	Discord, Notion, Plastic SCM

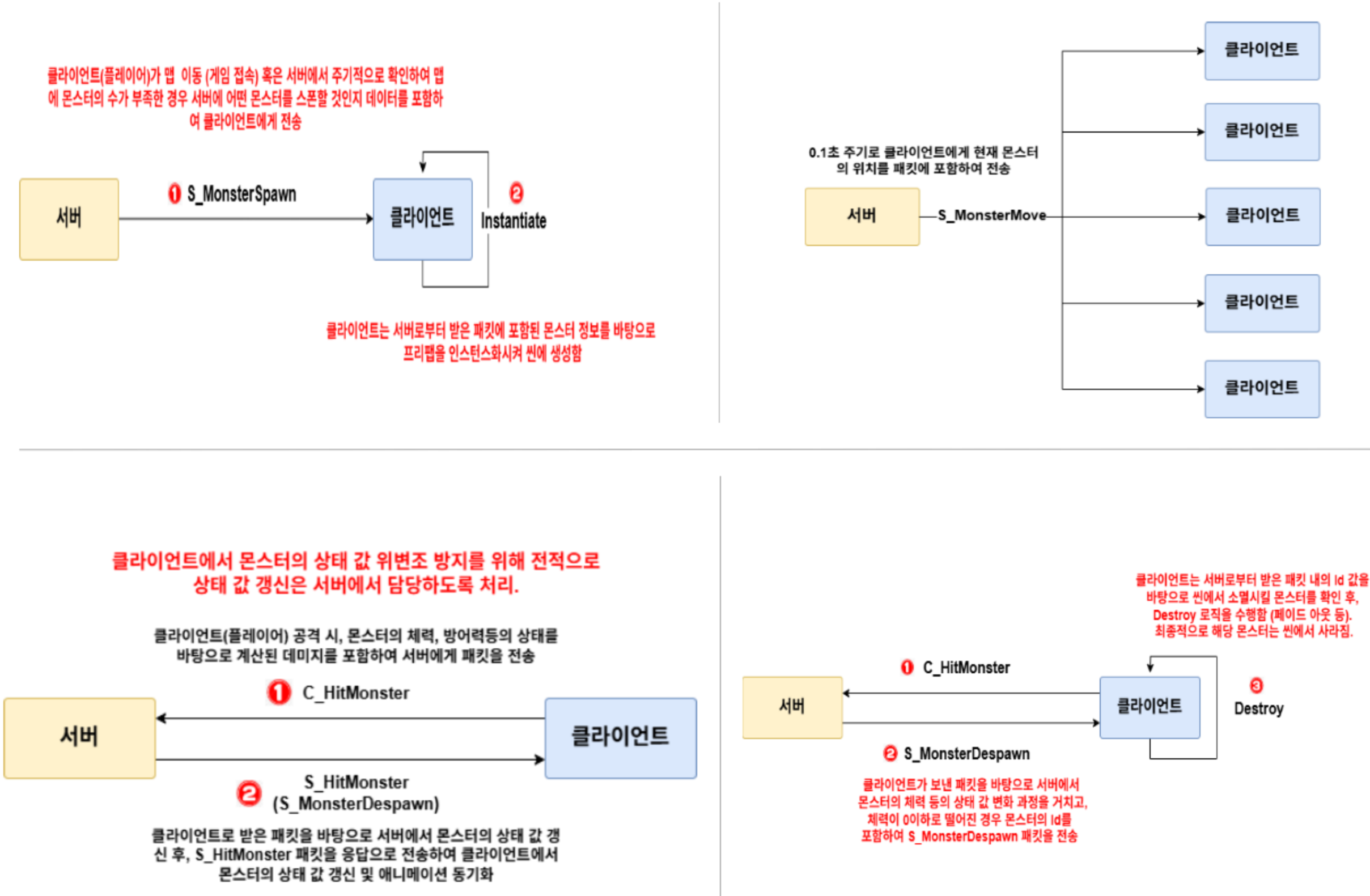
참고 링크 (코드, 협업, 플레이영상)



핵심 구현 1

서버와 클라이언트 몬스터 동기화

몬스터 동기화 관련 로직



MapleLike

[Unity 2D] 메이플스토리 모작 멀티플레이어 게임

기간

2024. 12 ~ 2025. 03

주관

실전 Unity 게임 개발 (멋쟁이 사자처럼)

참여인원

5명 (서버 개발자 1명 + 클라이언트 개발자 4명)

개발도구

언어	C#
클라이언트 (엔진)	Unity 6000.0.38f1
서버	Protobuf, AWS
협업	Discord, Notion, Plastic SCM

참고 링크 (코드, 협업, 플레이영상)

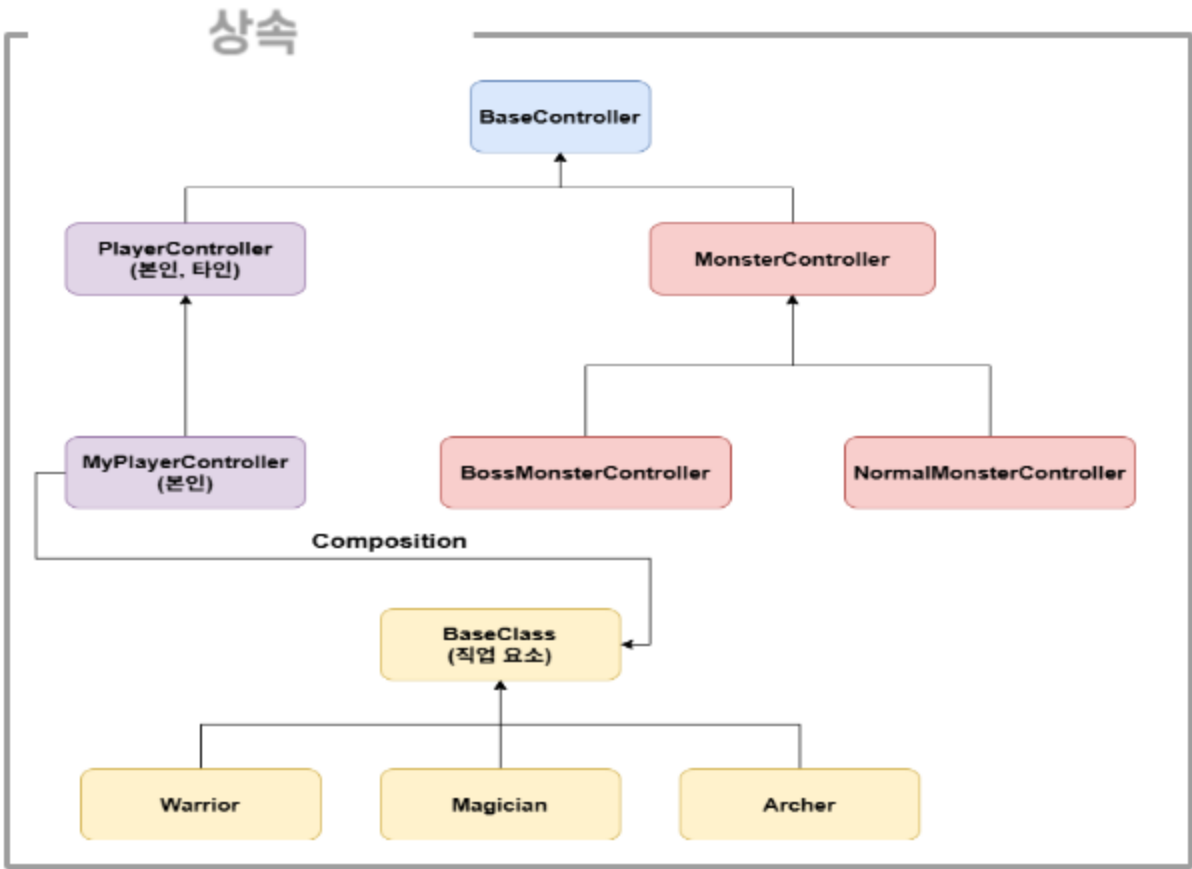


핵심 구현 2

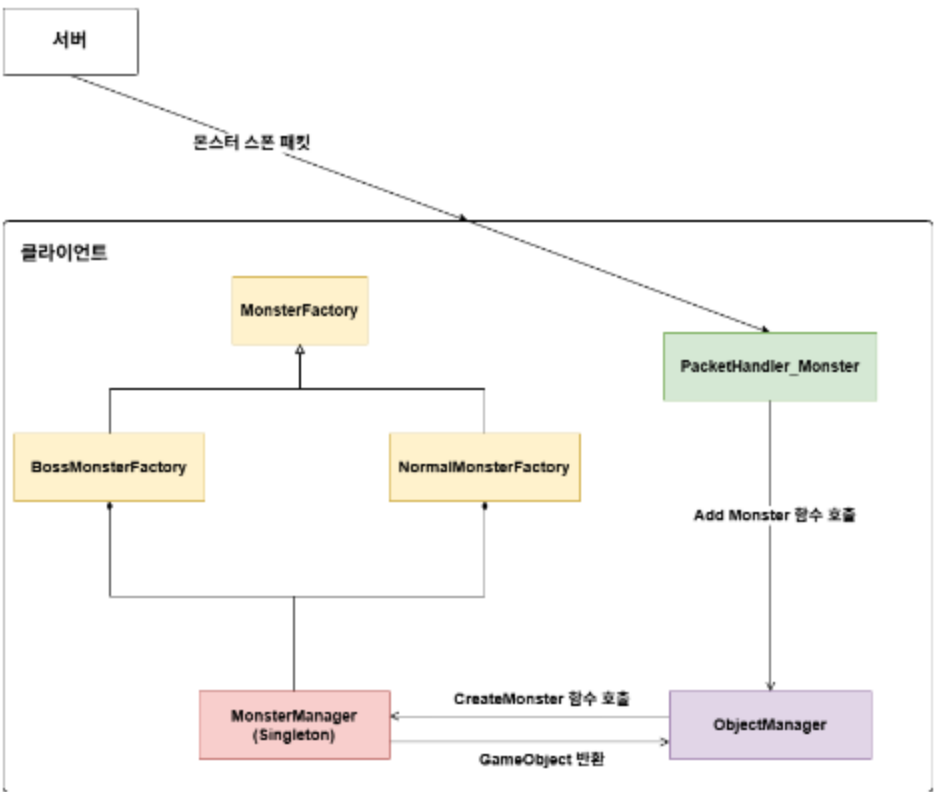
객체지향적 설계

- 기획 단계부터 구조적인 객체지향적 설계를 진행하였습니다.
 - **상속과 다형성**을 활용하여 중복 코드를 줄이고, 새로운 기능이나 몬스터 유형을 손쉽게 확장할 수 있도록 설계하였습니다.
 - 또한, **싱글턴, 스테이트, 팩토리**와 같은 **디자인 패턴**을 적극 활용하여 객체간 의존성을 최소화하고, 유연하고 확장 가능한 구조를 구축했습니다.

→ 이러한 객체지향적 접근을 통해 **불필요한 코드 작성을 줄이고 모듈 간 결합도를 낮춤으로써, 개발 기간을 단축하고 유지보수성을 크게 향상시킬 수 있었습니다.**



디자인 패턴



```
public void AddMonster(MonsterInfo info)
{
    GameObjectType objectType = GetObjectTypeById(info.MonsterId);

    if (objectType == GameObjectType.Normalmonster)
    {
        ...
        // NormalMonster 스폰 로직
    }
    else if (objectType == GameObjectType.Bossmonster)
    {
        ...
        // BossMonster 스폰 로직
    }
}
```

```
public void AddMonster(MonsterInfo info)
{
    GameObject monster = MonsterManager.Instance.CreateMonster(info);
    if (monster == null) return;

    // 추가된 몬스터를 객체 관리에 추가
    _objects.Add(info.MonsterId, monster);
}
```

→ 몬스터 생성 로직을 팩토리 패턴 기반의 MonsterManager로 분리해, ObjectManager의 책임을 줄이고 구조를 유연하게 개선했습니다.

MapleLike

[Unity 2D] 메이플스토리 모작 멀티플레이어 게임

기간

2024. 12 ~ 2025. 03

주관

실전 Unity 게임 개발 (멋쟁이 사자처럼)

참여인원

5명 (서버 개발자 1명 + 클라이언트 개발자 4명)

개발도구

언어	C#
클라이언트 (엔진)	Unity 6000.0.38f1
서버	Protobuf, AWS
협업	Discord, Notion, Plastic SCM

참고 링크 (코드, 협업, 플레이영상)



문제해결 1

씬 전환 시 패킷처리 문제

씬 전환 시 패킷 처리

- 클라이언트가 씬을 전환하는 과정(씬 로드 중)에서도 서버로부터의 패킷은 지속적으로 수신됩니다.
- 문제 상황
 - 씬 로드 도중, 몬스터나 플레이어처럼 네트워크 동기화가 필요한 오브젝트에 대한 서버 패킷이 수신되었지만, 클라이언트에서는 해당 오브젝트의 세팅이 아직 완료되지 않아, 패킷 처리 로직이 정상적으로 수행되지 않는 문제가 발생하였습니다.
- 문제 해결
 - 서버 개발자와 협업하여 디버깅을 진행, 문제의 원인을 파악하고 새로운 해결책으로, 씬 로딩 중 수신되는 패킷을 임시로 저장할 수 있는 큐(일종의 버퍼)를 도입하기로 결정하였습니다.
 - 이 큐는 씬이 완전히 로드될 때까지 서버로부터 전달되는 모든 패킷을 보관하며, 씬 로딩이 완료된 시점에서 해당 패킷들을 순차적으로 처리하도록 구현을 변경함으로써 문제를 해결하였습니다.

```
// SceneManager.cs (클라이언트)

// 씬이 로딩 중이면 대기열 큐에 추가, 아니면 즉시 실행
public void ExecuteOrQueue(System.Action action)
{
    if (IsSceneLoading)
    {
        pendingActions.Enqueue(action);
    }
    else
    {
        action.Invoke();
    }
}
```

```
// PacketHandler.cs
public static void S_MonsterSpawnHandler(PacketSession session, IMessage packet)
{
    S_MonsterSpawn spawnPacket = packet as S_MonsterSpawn;

    SceneManager.Instance.ExecuteOrQueue(() =>
    {
        foreach (MonsterInfo obj in spawnPacket.MonsterInfos)
        {
            ObjectManager.Instance.AddMonster(obj);
        }
    });
}
```

MapleLike

[Unity 2D] 메이플스토리 모작 멀티플레이어 게임

기간

2024. 12 ~ 2025. 03

주관

실전 Unity 게임 개발 (멋쟁이 사자처럼)

참여인원

5명 (서버 개발자 1명 + 클라이언트 개발자 4명)

개발도구

언어	C#
클라이언트 (엔진)	Unity 6000.0.38f1
서버	Protobuf, AWS
협업	Discord, Notion, Plastic SCM

참고 링크 (코드, 협업, 플레이영상)



문제해결 2

유니티 오브젝트 라이프 사이클

오브젝트 라이프 사이클

- MonoBehaviour를 상속받는 클래스는 Unity의 오브젝트 라이프 사이클을 따릅니다. 이 라이프 사이클을 충분히 고려하지 않은 구현은 비동기 네트워크 환경에서 다양한 문제가 발생할 수 있습니다.
- 문제 상황
 - 서버로부터 MonsterSpawn 패킷을 수신한 후, 몬스터 오브젝트가 인스턴스화되었지만, 필요한 컴포넌트가 변수에 아직 저장되기 전 상태에서 다른 클라이언트의 공격으로 인해 해당 몬스터가 사망하였고, 이에 따라 서버로부터 MonsterDespawn 패킷을 수신하여 Despawn 로직이 실행되었습니다.
 - 이 과정에서, 컴포넌트가 저장되지 않은 변수에 접근하면서 NullReferenceException이 발생하였습니다. 결과적으로 서버 기준에서는 몬스터가 정상적으로 소멸되었으나, 클라이언트에서는 해당 몬스터 오브젝트가 제거되지 않고 화면에 남아, 플레이어에게 혼란을 유발하는 문제로 이어졌습니다.
- 문제 해결
 - 이 문제는 몬스터에만 국한된 것이 아니라, 유니티 오브젝트의 초기화 타이밍을 올바르게 다루지 않을 경우 다른 오브젝트에서도 반복될 수 있는 공통된 이슈로 인지하였습니다.
 - 따라서, 팀원들과 함께 유니티 오브젝트 라이프 사이클에 대한 학습을 진행하였고, 컴포넌트 변수 할당을 오브젝트 생성 직후 가장 빠른 시점 (예: Awake 또는 OnEnable)에 수행하도록 구현을 개선하였습니다.
 - 추가적으로, 컴포넌트 초기화 여부를 확인할 수 있는 Boolean 플래그를 도입하여 안정성을 강화하였습니다.

```
// isInitialize boolean 변수의 사용과 함께,  
// 인스턴스화 후 바로 호출되는 Awake에서 컴포넌트가 변수가 저장되도록하여 NullReference 문제를 해결.  
protected void Awake()  
{  
    monsterRigidbody = GetComponent<Rigidbody2D>();  
    monsterCollider = GetComponent<Collider2D>();  
    monsterAnimator = GetComponent<Animator>();  
    monsterSpriteRenderer = GetComponent<SpriteRenderer>();  
    monsterAudioSource = GetComponent<AudioSource>();  
  
    isInitialized = true;  
  
    StartCoroutine(SpawnCoroutine());  
}
```

```
public void Remove(int id)  
{  
    GameObject go = FindById(id);  
    if (go == null)  
        return;  
  
    // 몬스터의 경우, 각 스크립트에서 Destroy 처리.  
    if (go.TryGetComponent<MonsterController>(out _))  
    {  
        StartCoroutine(SafeDespawnRoutine(go));  
    }  
    else Object.Destroy(go); // Unity 메인 스레드에서 오브젝트 삭제하고 _objects.Remove(id); // 딕셔너리에서 제거한다.  
}  
  
private IEnumerator SafeDespawnRoutine(GameObject go)  
{  
    if (!go.TryGetComponent<MonsterController>(out var mc))  
        yield break;  
  
    // 코루틴을 통해, isInitialized 변수의 값을 지속적으로 확인.  
    // 초기화가 완료되었을 경우에만, Despawn로직을 수행하도록 하여 추가적인 안정성 확보.  
    while (!mc.isInitialized)  
        yield return null;  
  
    // 죽는 애니메이션 동기화  
    ...  
  
    mc.Despawn();  
}
```